The algorithm is:

1. Cluster data into k groups, where k is predefined.
2. Select random cluster centroids from the k points.
3. Allocate data to the nearest clusters according to the Euclidean distance calculations.
4. Calculate new cluster centroids for next data.
5. Repeat steps from 2 to 4 until no further clustering is possible.

Reinforcement learning

This type of learning works based on a reward/penalty policy and the agent is programmed to do certain predefined functions in an environment. After executing the functions, the agent gets feedback from the environment, either as a reward or penalty. This changes the state of the environment and the agent also gathers the state of the environment as an input. Depending on the state and reward values, the agent takes decisions. Depending on the rewards and state changes, the agent improves its quality of performance. This type of learning approach is used in service robots to train them in certain environments and it doesn't require any training data sets or data models. Here, the Q-learning algorithm is used and works with the following formula:

$$Q_{new}(s_t, a_t) \leftarrow Q_{old}(s_t, a_t) + \alpha.(r_t + y.\ max_a Q(s_{t+1}, a) - Q_{old}(s_t, a_t))$$

Here,

- Q_{new} is the new state of the agent
- s_t is the state of the agent in time t
- a_t is the action taken by the agent in time t.
- α is the learning rate
- r_t is the reward in time t
- γ is the discount factor

Example: If the agent takes actions (A1,A2,A3,A4), causes state changes (S1,S2,S3,S4) and receives a set of rewards R, then the Q values will be initialised and the rewards data will be created as follows:

Q =	States	A1	A2	A3	A4
	S1	0	0	0	0
	S2	0	0	0	0
	S3	0	0	0	0
	S4	0	0	0	0

R =	States	A1	A2	A3	A4
	S1	0		0	1
	S2	0	0		0
	S3		0	0	1
	S4	0		0	1

Here, the reward values are either 1 or 0 based on the actions.

If the agent is in state S4, then there are three possible actions that can change the state to S1, S3, and S4. This will be calculated using a simplified formula:

```
Q(sate, action) = R(state, action) + y × Max[Q(next state, all actions)]
Q(S1, A4) = R(S1, A4) + 0.8 × Max[Q(S4, A1),( S4, A3),( S4, A4)]
          = 1 + 0.8 × 0 = 1
```

Here, the value of *Max[Q(next state, all actions)]* is zero, because the Q values were initialised to zero.

So, as the new Q value for *Q(S1, A4)* is 1, the Q values will be updated:

Q =	States	A1	A2	A3	A4
	S1	0	0	0	1
	S2	0	0	0	0
	S3	0	0	0	0
	S4	0	0	0	0

Similarly, the next states are determined by the agent, and the Q values will keep on updating.

The algorithm is:

1. Agent starts in state (s_t) and Q values are initialised.
2. Take action a_t and wait for a reward

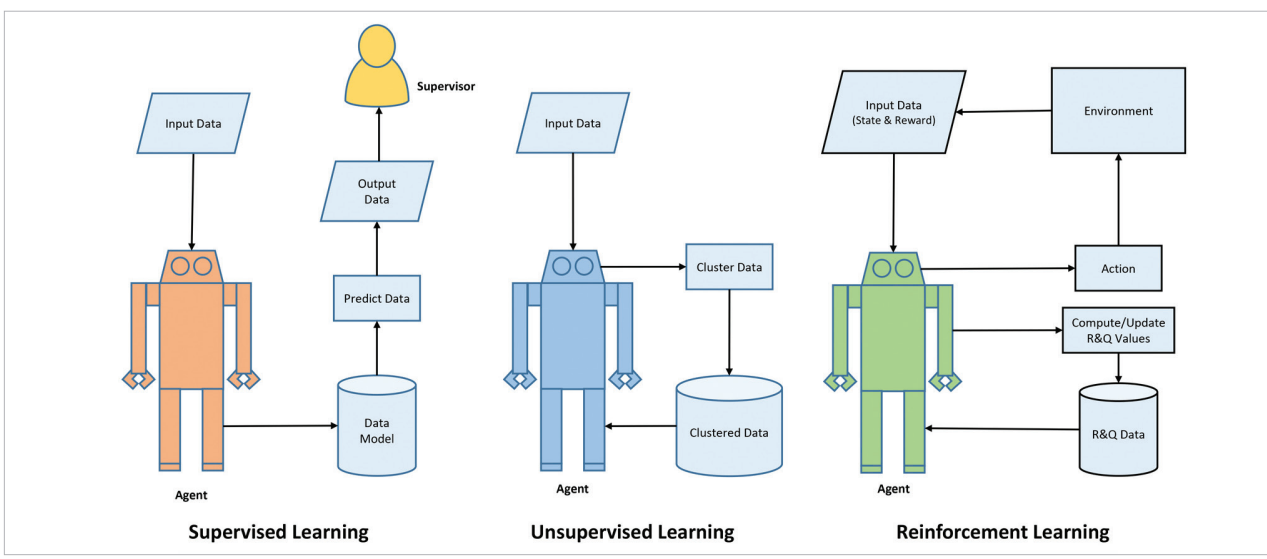

Figure 1: Types of learning approaches